LAPORAN UJIAN TENGAH SEMESTER
PRAKTIKUM PEMROGRAMAN MOBILE

GET FIT GYM: APLIKASI KATALOG LATIHAN BERBASIS FLUTTER DAN EXERCISEDB API

Disusun oleh:

Faril Isra Albiansyah

240605110087

PROGRAM STUDI TEKNIK INFORMATIKA

FAKULTAS SAINS DAN TEKNOLOGI

UNIVERSITAS ISLAM NEGERI MAULANA MALIK IBRAHIM MALANG

2026

BAB I. PENDAHULUAN

1.1 Latar Belakang dan Gambaran Aplikasi

Aktivitas kebugaran memerlukan informasi latihan yang mudah ditemukan dan dipahami. Pengguna pemula sering memerlukan petunjuk mengenai nama gerakan, kelompok otot yang dilatih, serta contoh visual untuk melakukan gerakan secara lebih tepat. Katalog latihan berbasis perangkat seluler dapat membantu pengguna menelusuri informasi tersebut dengan cepat.

GET FIT GYM edisi UTS dikembangkan menggunakan Flutter dan Dart sebagai aplikasi katalog latihan yang menampilkan data dinamis dari ExerciseDB OSS API. Informasi latihan disajikan melalui daftar latihan, pencarian, filter kategori atau peralatan, halaman detail, dan pratinjau demonstrasi gerakan dalam format GIF apabila media tersedia. Aplikasi ini berfokus pada pemenuhan capaian praktikum berupa HTTP request, pengolahan data JSON, asynchronous programming, dan antarmuka pengguna.

Sasaran aplikasi adalah mahasiswa atau pengguna umum yang ingin mengenal variasi gerakan latihan. Pada versi UTS, ruang lingkup dibatasi pada eksplorasi latihan; fitur yang lebih kompleks seperti pencatatan riwayat latihan, autentikasi, dan penjadwalan tidak dijadikan fokus utama.

1.2 Fitur Aplikasi

Fitur	Deskripsi dan manfaat
Beranda	Menampilkan identitas aplikasi dan pintasan ke katalog latihan.
All Exercise	Memuat daftar latihan dari API secara bertahap.
Kategori target otot	Mengelompokkan latihan, antara lain Chest dan Back, untuk mempermudah eksplorasi.
Pencarian dan filter	Menyaring daftar yang telah dimuat berdasarkan istilah atau jenis peralatan; cakupan pencarian ditampilkan secara eksplisit.
Muat Lebih Banyak	Mengambil halaman data berikutnya tanpa menghilangkan daftar sebelumnya.
Preview GIF	Menampilkan demonstrasi visual gerakan jika media tersedia, dengan fallback jika gagal.
Status permintaan	Menyediakan mekanisme pemuatan, tampilan data, dan penanganan kegagalan permintaan.

BAB II. PEMBAHASAN

2.1 Persiapan Project dan Package

Project dikembangkan dengan Flutter SDK dan bahasa Dart pada lingkungan Visual Studio Code. Pada tahap implementasi API, kode dipisahkan menjadi komponen model, service, serta halaman/view, agar proses mengambil data dan menyajikan hasil dapat dipahami secara terstruktur.

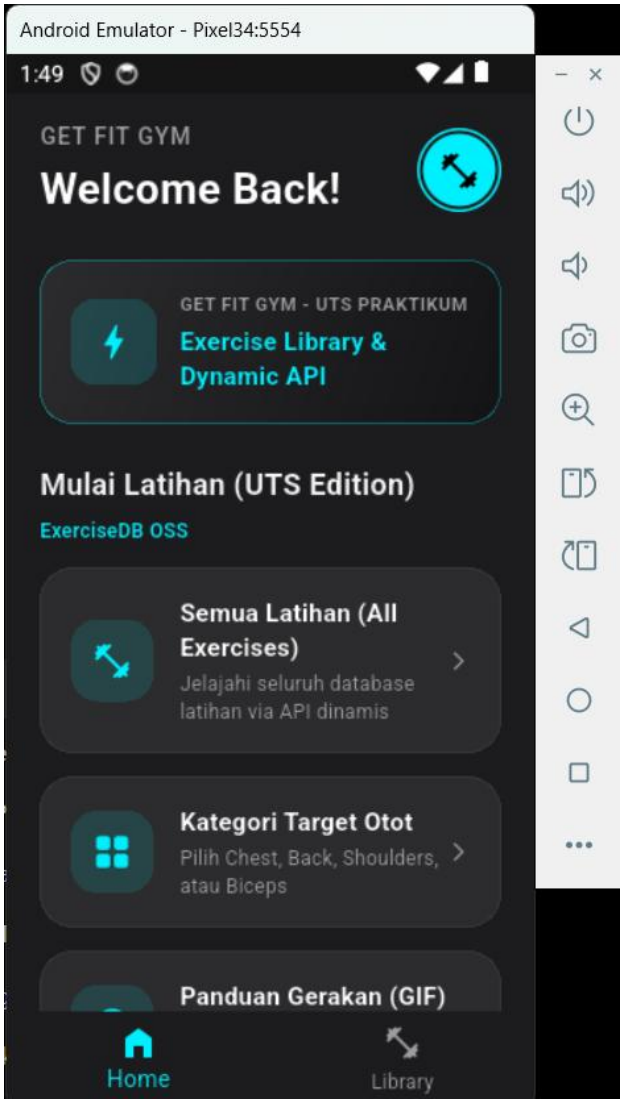
Komponen yang terlihat atau dilaporkan dalam project antara lain: lib/model/exercise_api_model.dart untuk pemetaan respons, lib/service/exercise_api_service.dart untuk permintaan HTTP, dan halaman daftar serta detail latihan pada lib/view/. Widget khusus digunakan untuk pratinjau GIF dan penerapan tampilan yang konsisten.

Teknologi / package	Kegunaan
Flutter / Dart	Pengembangan UI lintas platform dan logika aplikasi.
http	Melakukan permintaan HTTP ke layanan ExerciseDB OSS.
GetX (sudah ada pada project)	Mendukung navigasi atau pengelolaan state pada komponen lama; penggunaan aktual disesuaikan kode final.
Android SDK / Gradle	Membangun dan menjalankan aplikasi pada emulator Android.

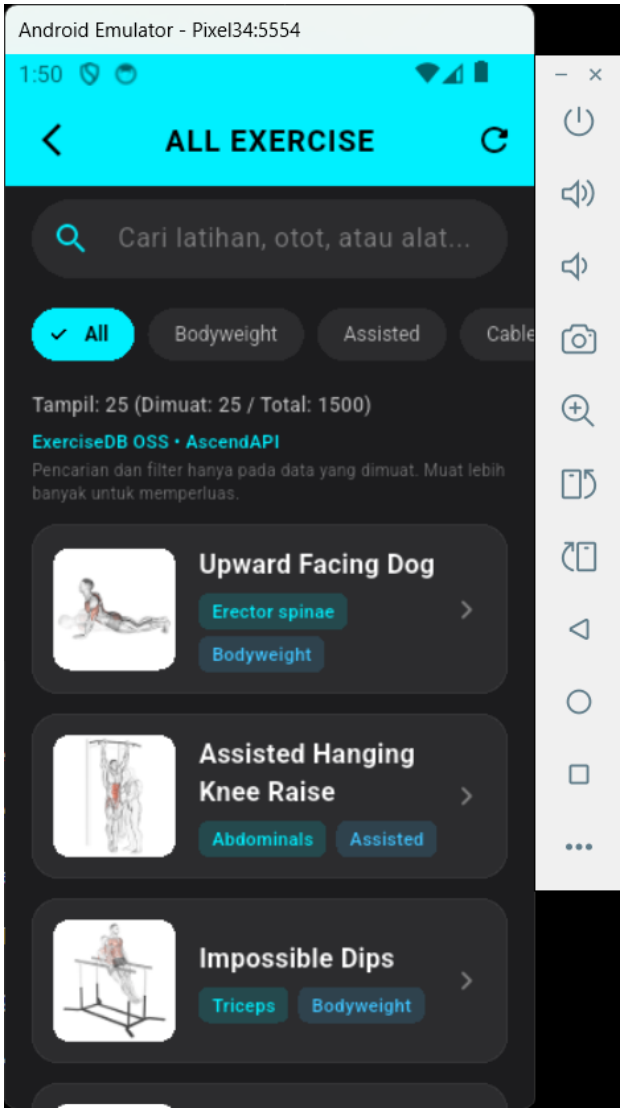
Perintah persiapan dan validasi yang digunakan meliputi flutter pub get, flutter analyze, flutter test, flutter build apk --debug, dan flutter run. Versi SDK/package yang tepat harus disalin dari hasil flutter --version dan pubspec.yaml pada source final.

2.2 Desain Antarmuka Pengguna (User Interface)

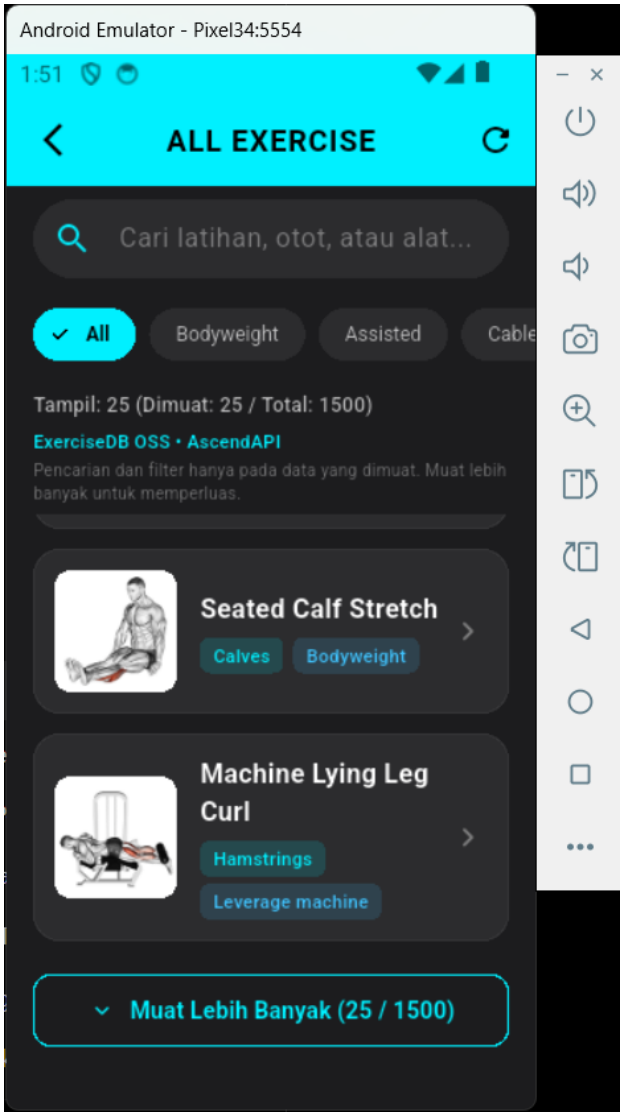
GET FIT GYM menggunakan tema gelap dengan warna aksen sian yang kontras. Halaman beranda menyajikan navigasi menuju katalog, sedangkan halaman daftar memanfaatkan kartu latihan yang berisi gambar/preview, nama latihan, otot target, dan jenis peralatan. Antarmuka menyediakan kolom pencarian, chip filter, informasi jumlah data, dan tombol memuat data tambahan.



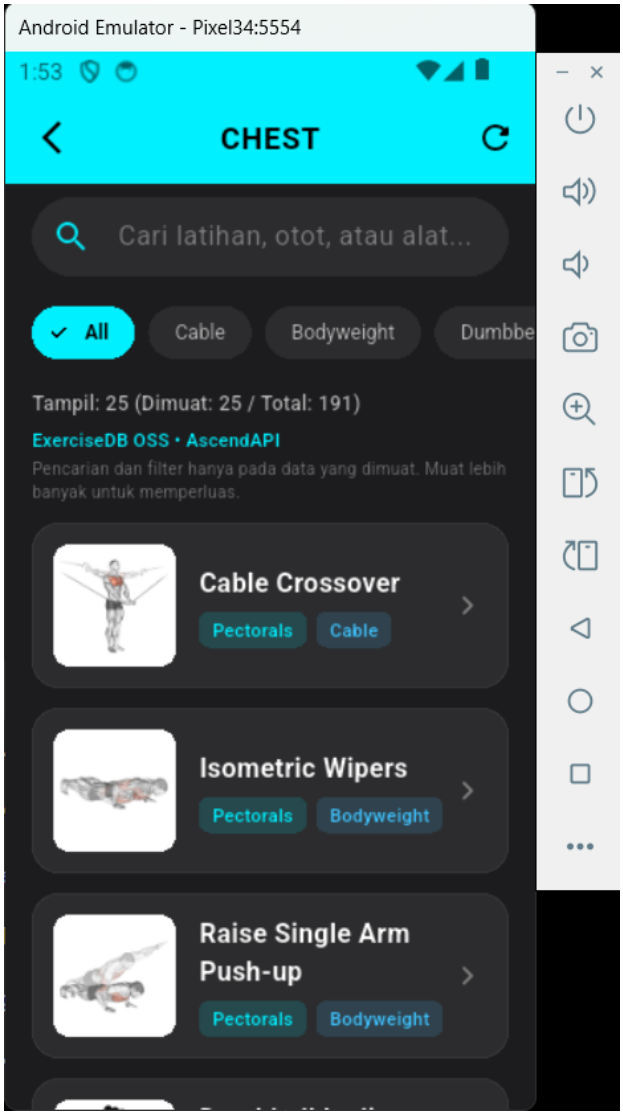
Gambar 2.1 Halaman beranda GET FIT GYM edisi UTS.



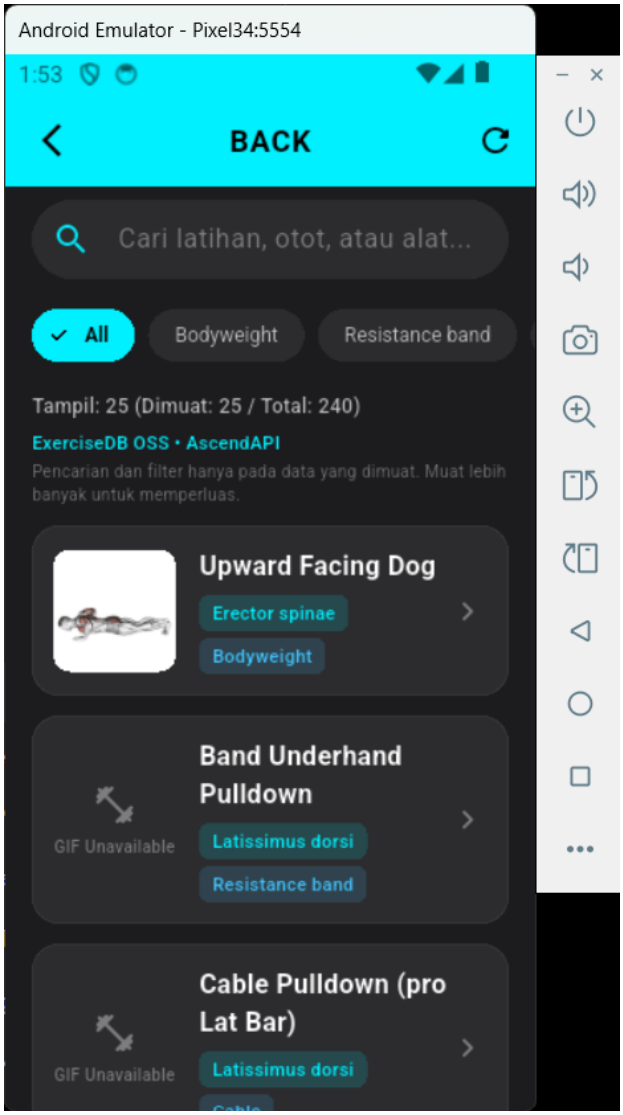
Gambar 2.2 Daftar All Exercise; pada layar terlihat 25 latihan dimuat dari total 1.500 yang ditampilkan aplikasi.



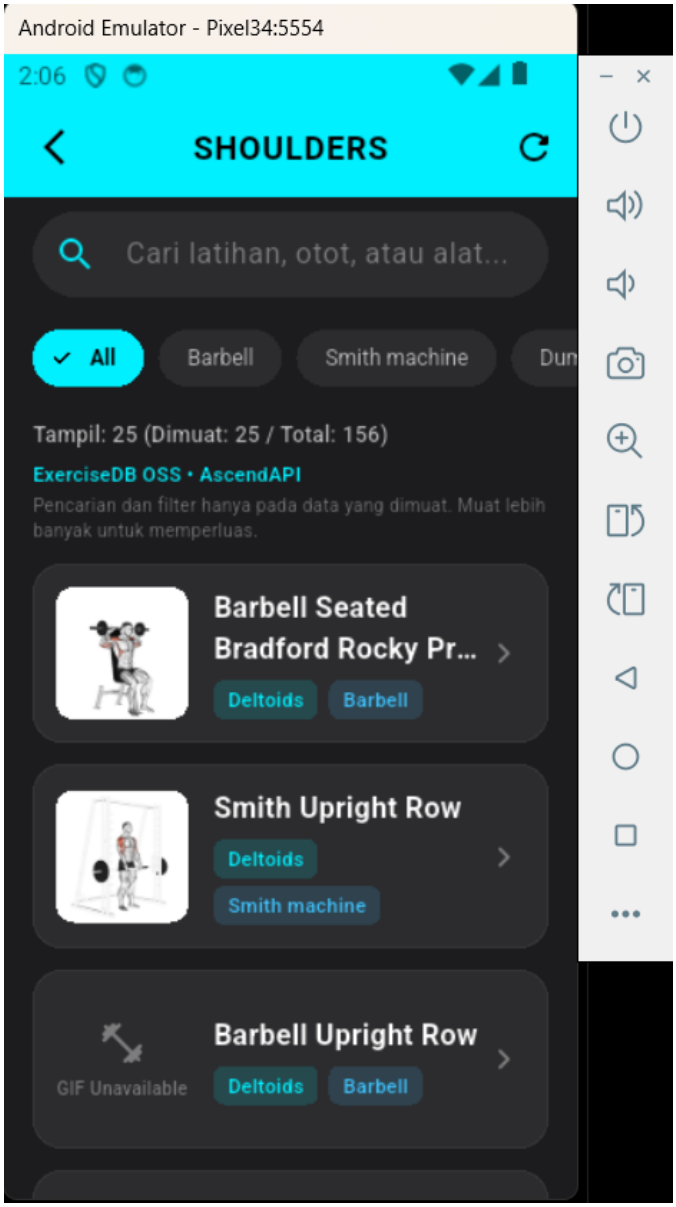
Gambar 2.3 Tombol "Muat Lebih Banyak" untuk mengambil halaman latihan berikutnya.



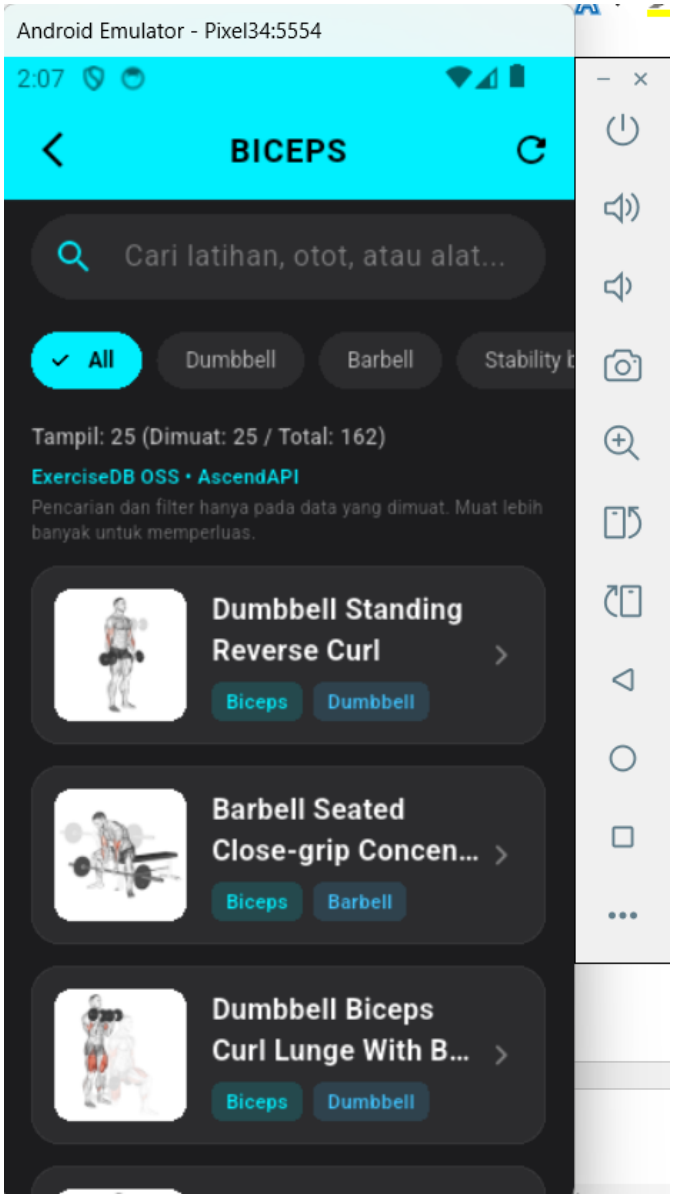
Gambar 2.4 Kategori Chest; aplikasi menampilkan total 191 latihan pada kategori ini.



Gambar 2.5 Kategori Back; sebagian GIF tersedia dan sebagian menampilkan fallback "GIF Unavailable".



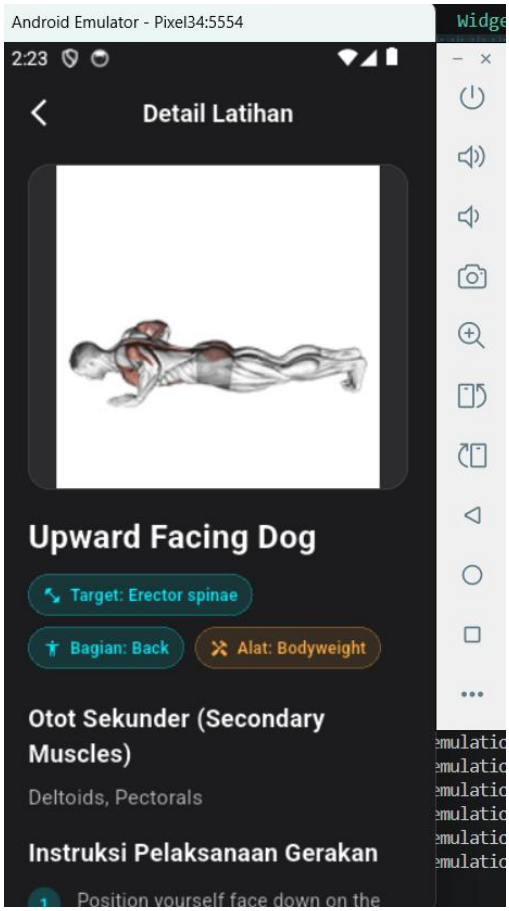
Gambar 2.6 Kategori Shoulders; sebagian GIF tersedia dan sebagian menampilkan fallback "GIF Unavailable"



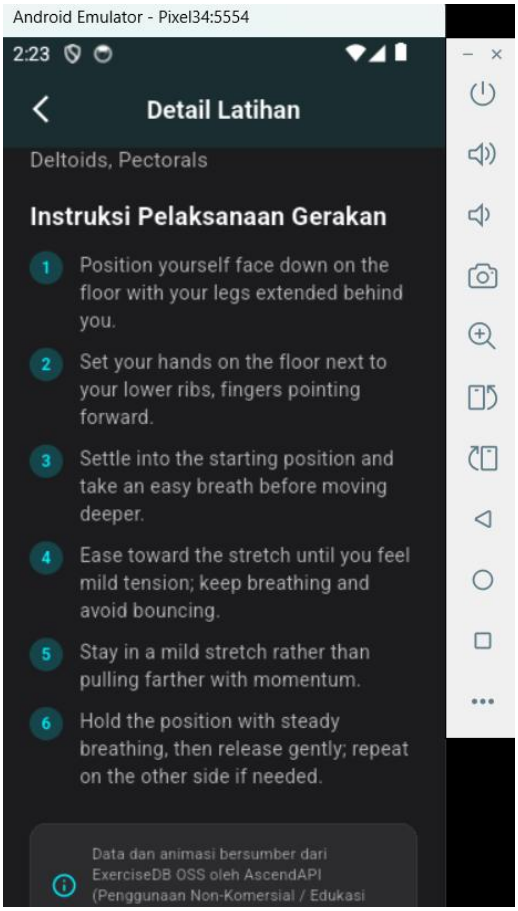
Gambar 2.7 Kategori Biceps; aplikasi menampilkan total 162 latihan pada kategori ini

Berdasarkan pengamatan screenshot, katalog utama menampilkan 25 data dari total 1.500; kategori Chest menampilkan 25 dari 191, kategori Back menampilkan 25 dari 240, kategori Shoulders menampilkan 25 dari 156, dan kategori Biceps menampilkan 25 dari 162 . Angka tersebut merupakan informasi yang ditampilkan antarmuka saat pengambilan gambar, bukan hasil audit seluruh data dari server.

Halaman Detail Latihan menampilkan demonstrasi gerakan, nama latihan, otot target, bagian tubuh, dan peralatan. Halaman ini juga menyajikan otot sekunder serta petunjuk langkah demi langkah sehingga pengguna dapat memahami gerakan yang dipilih. Pada contoh Upward Facing Dog, informasi target adalah Erector spinae, bagian tubuh Back, dan peralatan Bodyweight.



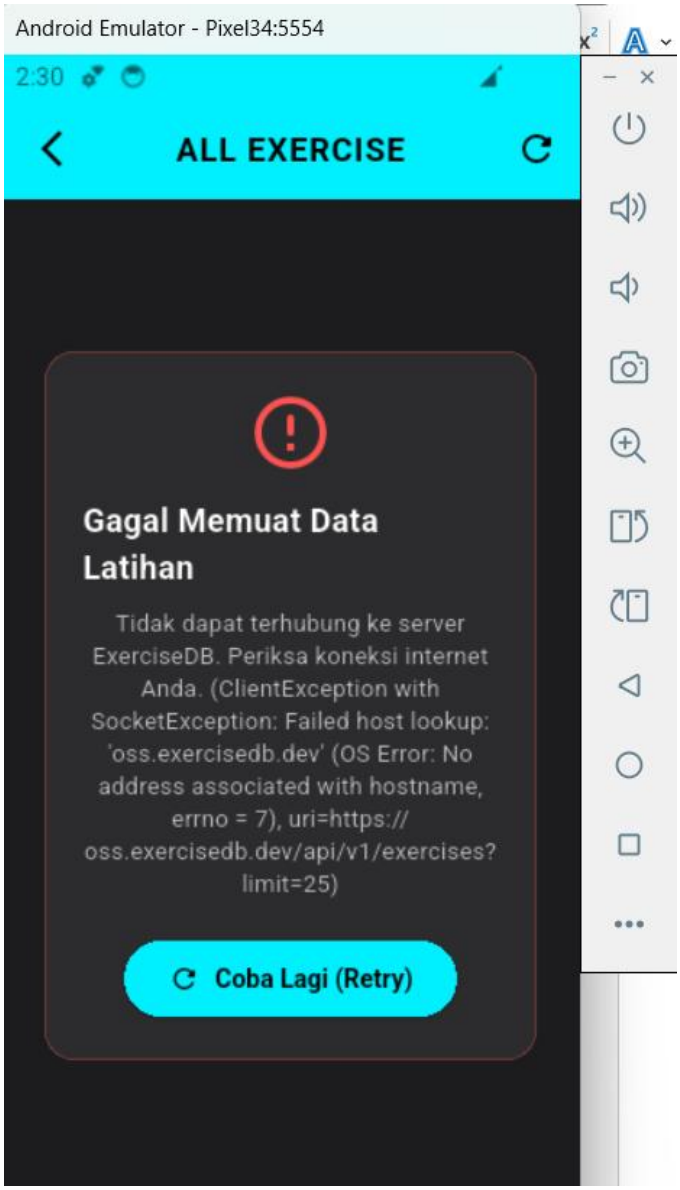
Gambar 2.8 Halaman Detail Latihan Upward Facing Dog menampilkan demonstrasi, otot target, bagian tubuh, dan peralatan.



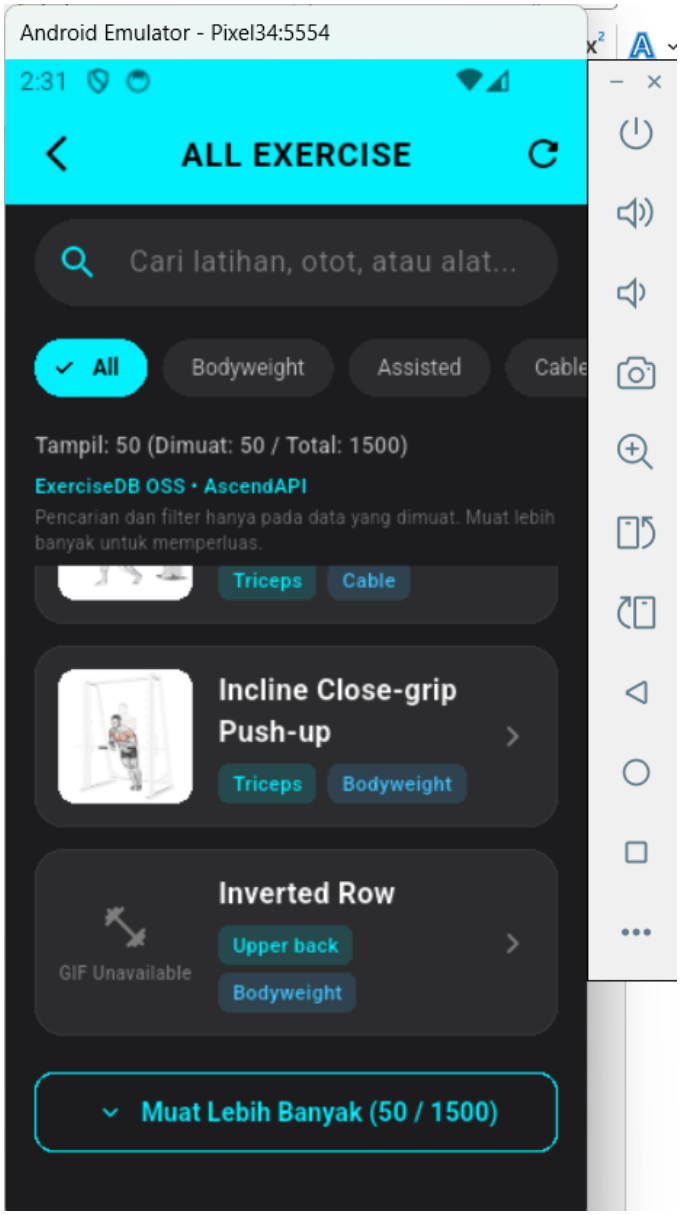
Gambar 2.9 Halaman Detail Latihan menampilkan instruksi gerakan bertahap serta atribusi sumber data dan animasi.

Kedua tampilan tersebut memperlihatkan bahwa data yang diperoleh dari API tidak hanya disajikan sebagai daftar, tetapi juga menjadi informasi detail dan instruksi latihan. Cuplikan layar merupakan bukti tampilan statis; animasi GIF yang bergerak tetap perlu diverifikasi langsung saat aplikasi dijalankan.

Pengujian tambahan dilakukan dengan memutus koneksi jaringan untuk mengamati penanganan kesalahan API dan mengaktifkan kembali koneksi sebelum mencoba ulang. Pengujian pagination juga dilakukan dengan menekan tombol “Muat Lebih Banyak” pada daftar All Exercise. Kedua kondisi tersebut didokumentasikan pada Gambar 2.10 dan Gambar 2.11.



Gambar 2.10 Tampilan error ketika koneksi ke ExerciseDB gagal, disertai tombol "Coba Lagi (Retry)".



Gambar 2.11 Hasil pagination: jumlah data All Exercise meningkat menjadi 50 dari total 1.500 setelah memuat halaman berikutnya.

2.3 Implementasi Pengambilan dan Pengolahan Data dari API

Aplikasi GET FIT GYM menggunakan ExerciseDB OSS API sebagai sumber data latihan olahraga. Layanan ini menyediakan informasi mengenai nama latihan, bagian tubuh yang dilatih, kelompok otot, peralatan, instruksi gerakan, dan URL animasi GIF apabila tersedia.

API diakses melalui alamat dasar https://oss.exercisedb.dev/api/v1 dengan endpoint /exercises. Proses pengambilan data dilakukan melalui permintaan HTTP GET. Data yang diterima dalam format JSON kemudian dikonversi menjadi objek Dart dan ditampilkan pada antarmuka aplikasi.

Implementasi API dipisahkan menjadi beberapa komponen, yaitu service untuk komunikasi jaringan, model untuk pengolahan data, dan halaman antarmuka untuk menampilkan informasi latihan.

2.3.1 Implementasi Pengambilan Data Menggunakan HTTP Request

Proses pengambilan data API diimplementasikan dalam berkas lib/service/exercise_api_service.dart melalui class ExerciseApiService.

Kode Program 2.1 Implementasi HTTP Request ke ExerciseDB OSS

```
final uri = Uri.parse('$baseUrl/exercises')
    .replace(queryParameters: queryParams);


try {
  final response = await _client.get(
    uri,
    headers: {
      'Accept': 'application/json',
      'User-Agent': 'GetFitGym-UTS/1.0',
    },
  ).timeout(defaultTimeout);


  return _handleResponse(response);
}
```

Potongan kode tersebut menunjukkan penggunaan await untuk menunggu respons dari server tanpa menghentikan seluruh proses aplikasi. Permintaan dikirimkan menggunakan package http dengan metode GET. Parameter pada queryParams digunakan untuk mengatur jumlah latihan, filter, dan pagination.

Aplikasi juga menetapkan batas waktu permintaan selama 15 detik. Apabila terjadi kendala jaringan, timeout, atau respons yang tidak sesuai, service akan menghasilkan exception dengan pesan kesalahan yang dapat ditampilkan kepada pengguna.

Kode Program 2.2 Validasi Status HTTP dan Dekode JSON

```
if (statusCode >= 200 && statusCode < 300) {
  final dynamic decoded = json.decode(response.body);


  if (decoded is! Map<String, dynamic>) {
    throw const ApiException(
      'Format data JSON dari server tidak sesuai standar.',
```

```
    );
  }

  final success = decoded['success'] as bool? ?? false;
  if (!success) {
    throw ApiException(
      'Server ExerciseDB merespons status gagal.',
      statusCode: statusCode,
    );
  }
}
```

Kode tersebut memeriksa apakah server mengembalikan status HTTP pada rentang 200–299. Jika berhasil, respons JSON diproses menggunakan json.decode(). Aplikasi kemudian memvalidasi struktur JSON serta nilai atribut success sebelum melakukan pengolahan data berikutnya.

Selain itu, service menangani respons HTTP 429 ketika batas permintaan API tercapai serta respons 500 ke atas ketika server mengalami kendala.

2.3.2 Konversi Respons JSON Menjadi Model Dart

Data JSON dari API dikonversi menjadi objek Dart menggunakan class ExerciseApiModel yang terdapat pada berkas lib/model/exercise_api_model.dart.

Kode Program 2.3 Konversi JSON Menggunakan Factory Constructor

```
factory ExerciseApiModel.fromJson(Map<String, dynamic> json) {
  return ExerciseApiModel(
    exerciseId: json['exerciseId'] as String? ?? '',
    name: json['name'] as String? ?? 'Unnamed Exercise',
    gifUrl: json['gifUrl'] as String? ?? '',
    bodyParts: _parseStringList(json['bodyParts']),
    equipments: _parseStringList(json['equipments']),
    targetMuscles: _parseStringList(json['targetMuscles']),
    secondaryMuscles: _parseStringList(json['secondaryMuscles']),
    instructions: _parseStringList(json['instructions']),
  );
```

```
}
```

Method fromJson() digunakan untuk mengubah setiap entri JSON menjadi objek ExerciseApiModel. Objek tersebut memiliki atribut seperti exerciseId, name, gifUrl, bodyParts, equipments, targetMuscles, secondaryMuscles, dan instructions.

Operator ?? digunakan untuk memberikan nilai alternatif ketika atribut tertentu bernilai null. Sementara itu, fungsi _parseStringList() membantu memastikan atribut yang berbentuk daftar dikonversi menjadi List<String>.

Dengan pendekatan ini, data latihan dapat diakses melalui properti objek Dart sehingga lebih mudah digunakan pada komponen antarmuka.

2.3.3 Penerapan Asynchronous Programming Menggunakan FutureBuilder

Aplikasi menerapkan asynchronous programming agar proses pengambilan data dari server tidak menghambat tampilan antarmuka pengguna.

Implementasinya terdapat pada halaman lib/view/list_all_exercise_page.dart menggunakan FutureBuilder<ExerciseApiResponse>.

Kode Program 2.4 Implementasi FutureBuilder

```
body: FutureBuilder<ExerciseApiResponse>(
 future: _exercisesFuture,
 builder: (context, snapshot) {
  if (snapshot.connectionState == ConnectionState.waiting) {
   return _buildLoadingState();
  }

  if (snapshot.hasError && _loadedExercises.isEmpty) {
   final errorMsg = snapshot.error is ApiException
       ? (snapshot.error as ApiException).message
       : snapshot.error.toString();
   return _buildErrorState(errorMsg);
  }

  if (_loadedExercises.isEmpty) {
   return _buildEmptyState(
    'Tidak ada data latihan yang diterima dari server.',
```

```
    );
  }

  final filteredExercises = _filterExercises(_loadedExercises);
```

Potongan tersebut menunjukkan penanganan beberapa kondisi pengambilan data. Ketika permintaan masih berlangsung, aplikasi menampilkan loading indicator melalui _buildLoadingState(). Jika terjadi kesalahan, aplikasi memanggil _buildErrorState() untuk menampilkan informasi kegagalan dan tombol mencoba kembali.

Apabila tidak terdapat latihan yang diterima dari server, aplikasi menampilkan empty state. Sebaliknya, ketika data tersedia, aplikasi melakukan penyaringan dan membentuk daftar kartu latihan menggunakan ListView.builder.

Future disimpan pada variabel _exercisesFuture yang diinisialisasi melalui proses pemuatan data. Pendekatan tersebut memungkinkan data dipertahankan selama proses pembangunan ulang widget yang tidak memerlukan permintaan baru.

2.3.4 Implementasi Pagination dan Fitur Muat Lebih Banyak

ExerciseDB menyediakan data latihan dalam jumlah besar. Oleh karena itu, aplikasi menggunakan mekanisme pagination agar data tidak harus diambil seluruhnya dalam satu permintaan.

Pada implementasi GET FIT GYM, data awal diminta sebanyak 25 latihan. Ketika pengguna memilih tombol "Muat Lebih Banyak", aplikasi meminta halaman berikutnya menggunakan cursor.

Kode Program 2.5 Implementasi Pagination

```
final res = await _apiService.fetchExercises(
  limit: 25,
  cursor: _nextCursor,
  bodyPart: _resolveBodyPartParam(),
  targetMuscle: _resolveTargetMuscleParam(),
);

setState(() {
  _loadedExercises.addAll(res.exercises);
  _nextCursor = res.nextCursor;
  _hasNextPage = res.hasNextPage;
  _totalServerCount = res.total;
```

```
  _isLoadingMore = false;
});
```

Kode tersebut terdapat pada fungsi _loadMore(). Parameter cursor digunakan untuk meminta data halaman berikutnya. Data yang berhasil diterima kemudian ditambahkan ke _loadedExercises menggunakan addAll() sehingga daftar sebelumnya tidak hilang.

Fungsi setState() memperbarui antarmuka setelah data baru diterima. Informasi nextCursor, hasNextPage, dan total digunakan untuk mengatur kelanjutan pagination serta menampilkan jumlah data.

Berdasarkan pengujian tampilan emulator, kategori All Exercise menunjukkan total 1.500 latihan, sedangkan kategori Chest dan Back masing-masing menunjukkan total 191 dan 240 latihan pada saat pengujian. Angka tersebut merupakan hasil yang ditampilkan aplikasi berdasarkan respons API pada waktu pengujian, bukan jumlah yang dianggap tetap.

Fitur pencarian dan filter peralatan pada implementasi ini bekerja terhadap data yang sudah dimuat. Pengguna perlu memuat halaman berikutnya untuk memperluas cakupan pencarian.

2.3.5 Implementasi Animasi GIF Latihan

Selain informasi teks, GET FIT GYM menampilkan demonstrasi gerakan olahraga menggunakan animasi GIF yang diperoleh melalui URL media pada data latihan.

Komponen tersebut diimplementasikan dalam class ExerciseGifWidget pada berkas widget penampil GIF.

Kode Program 2.6 Implementasi Penampilan GIF

```
child: Image.network(
  gifUrl,
  width: width,
  height: height,
  fit: fit,
  gaplessPlayback: true,
  loadingBuilder: (context, child, loadingProgress) {
    if (loadingProgress == null) {
      return child;
    }
```

Widget Image.network() digunakan untuk menampilkan media dari URL API. Properti gifUrl berasal dari objek latihan yang telah diproses oleh model Dart.

Ketika media masih dimuat, loadingBuilder menampilkan indikator progres. Apabila media gagal diakses, errorBuilder menampilkan placeholder bertuliskan "GIF Unavailable". Jika URL kosong atau tidak valid, widget juga menampilkan placeholder.

Penanganan tersebut diperlukan karena tidak semua entri latihan memiliki media yang dapat diakses. Pada hasil pengujian emulator, beberapa latihan menampilkan gambar demonstrasi, sedangkan latihan lain menampilkan informasi GIF tidak tersedia. Halaman detail pada Gambar 2.8 dan 2.9 juga menunjukkan penggunaan atribut targetMuscles, bodyParts, equipments, secondaryMuscles, dan instructions yang berasal dari model data.

Dengan implementasi tersebut, aplikasi dapat menyajikan informasi latihan secara dinamis, menampilkan demonstrasi visual apabila tersedia, dan tetap memberikan informasi yang jelas ketika media gagal dimuat.

Secara keseluruhan, penerapan HTTP request, validasi status server, konversi JSON ke model Dart, FutureBuilder, pagination, dan penampilan GIF menunjukkan penggunaan konsep integrasi API serta asynchronous programming sesuai dengan tujuan praktikum Pemrograman Mobile.

2.4 Pengujian Aplikasi (Pelengkap Hasil Implementasi)

Pengujian dilakukan untuk memeriksa proses kompilasi, kualitas kode, pemetaan JSON, perilaku HTTP service, kondisi antarmuka, dan pagination. Berdasarkan hasil terminal Hermes yang dikirimkan, flutter analyze tidak menemukan issue, sembilan pengujian otomatis lulus, dan APK debug berhasil dibangun. Hasil tersebut perlu disandingkan dengan log atau tangkapan layar terminal final pada lampiran jika diminta dosen.

Skenario	Hasil/bukti tersedia
Beranda terbuka	Terlihat pada screenshot emulator (Gambar 2.1).
API latihan termuat	25 data tampil pada All Exercise (Gambar 2.2).
Pagination	Berhasil bertambah dari 25 data (Gambar 2.2–2.3) menjadi 50 data yang dimuat (Gambar 2.11).
Kategori Chest	Kartu latihan dan total kategori terlihat (Gambar 2.4).
Kategori Back / fallback GIF	Kartu latihan serta fallback GIF terlihat (Gambar 2.5).
Kategori Shoulders / fallback GIF	Kartu latihan serta fallback GIF terlihat (Gambar 2.6).
Kategori Biceps	Kartu latihan dan total kategori terlihat (Gambar 2.7).

flutter analyze	0 error dan 0 warning.
flutter test	9 dari 9 pengujian lulus.
flutter build apk --debug	APK debug berhasil dibuat.
Detail latihan dan instruksi	Nama, target otot, bagian tubuh, peralatan, otot sekunder, dan instruksi bertahap terlihat pada Gambar 2.8–2.9.
Error jaringan dan Retry	Saat alamat ExerciseDB tidak dapat di-resolve karena kendala koneksi, aplikasi menampilkan pesan kegagalan dan tombol Coba Lagi (Retry) (Gambar 2.10). Tombol tersedia; hasil penekanan tombol tidak terdokumentasi dalam screenshot.
Pagination setelah tombol ditekan	Daftar All Exercise menunjukkan 50 data dimuat, meningkat dari 25, dengan total server 1.500 pada saat pengujian (Gambar 2.11).

Pengujian manual tambahan memperlihatkan bahwa pesan kegagalan koneksi dan tombol Retry muncul saat server tidak dapat diakses (Gambar 2.10). Selain itu, tombol Muat Lebih Banyak berhasil menambah jumlah data yang dimuat dari 25 menjadi 50 (Gambar 2.11). Pengujian lanjutan yang masih disarankan adalah memastikan GIF bergerak di emulator dan menguji hasil penekanan Retry setelah koneksi internet dipulihkan.

BAB III. PENUTUP

3.1 Kesimpulan

GET FIT GYM edisi UTS telah dikembangkan sebagai katalog latihan berbasis Flutter yang mengintegrasikan data ExerciseDB OSS API. Antarmuka emulator memperlihatkan beranda, daftar latihan dinamis, kategori otot, filter, fitur paginasi, serta penanganan media latihan yang tidak tersedia. Struktur aplikasi memisahkan tugas pengambilan data, pengolahan JSON, dan penyajian melalui antarmuka.

Berdasarkan laporan pengujian otomatis dari Hermes, analisis kode tidak menemukan masalah, sembilan pengujian lulus, dan APK debug berhasil dibangun. Pengujian manual melalui emulator juga memperlihatkan penanganan kegagalan jaringan dengan pesan kesalahan dan tombol Retry serta keberhasilan pagination meningkatkan data yang dimuat dari 25 menjadi 50. Pengujian animasi GIF bergerak dan keberhasilan Retry setelah koneksi pulih tetap dapat dilengkapi sebagai verifikasi tambahan.

3.2 Saran

Pengembangan berikutnya dapat meningkatkan pencarian server-side sehingga hasil tidak terbatas pada data yang telah dimuat, menambahkan cache media latihan untuk mengurangi pemakaian jaringan, dan meningkatkan informasi fallback ketika gambar/GIF tidak tersedia. Pengujian pada perangkat dengan variasi ukuran layar dan kondisi jaringan juga disarankan.

DAFTAR PUSTAKA

Flutter. (2026). Flutter documentation. https://docs.flutter.dev/ (diakses untuk implementasi Flutter dan widget).

Dart. (2026). Dart language documentation. https://dart.dev/guides (diakses untuk asynchronous programming dan pemodelan data).

pub.dev. (2026). http package. https://pub.dev/packages/http (referensi komunikasi HTTP pada Dart/Flutter).

ExerciseDB OSS. (2026). API documentation. https://oss.exercisedb.dev/docs (sumber API data latihan dan media).

uqifumi. (2026). Contoh Async Programming FutureBuilder. https://github.com/uqifumi (referensi materi praktikum).